

Lab10 Webclient

1.ทดสอบ Endpoint ที่ 6

1.1 ดึง Product ทั้งหมด

```
{
  "id": "1",
  "name": "iPhone 15 Pro (673380605-0 sec3)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900,
  "discountType": "MEMBER",
  "discountedPrice": 35910
},
{
  "id": "2",
  "name": "MacBook Air M3",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 20,
  "price": 49900,
  "discountType": "NONE",
  "discountedPrice": 49900
},
{
  "id": "3",
  "name": "Samsung Galaxy S24",
  "category": "Electronics",
  "brand": "Samsung",
  "stock": 30,
  "price": 29900,
  "discountType": "SEASONAL",
  "discountedPrice": 23920
}
}
```

1.2 ดึง Product 1 รายการตาม id

```
{
  "id": "1",
  "name": "iPhone 15 Pro (673380605-0 sec3)",
  "category": "Electronics",
  "brand": "Apple",
  "stock": 50,
  "price": 39900,
  "discountType": "MEMBER",
  "discountedPrice": 35910
}
```

1.3 สร้าง Product ใหม่

HTTP <http://localhost:8081/accounts> Save

POST <http://localhost:8080/products> Send

Params Auth Headers (8) **Body** Pre-req. Tests Settings Cookies

raw **JSON** Beautify

```
1 [{"id": "4", "name": "Ipad Air", "category": "Electronics", "brand": "Apple", "stock": 10, "price": 22900.0, "memberType": "NONE"}]
```

Body 200 OK 8 ms 214 B Save Response

Pretty Raw Preview Visualize **JSON**

```
1 {
2   "id": "4",
3   "name": "Ipad Air",
4   "category": "Electronics",
5   "brand": "Apple",
6   "stock": 10,
7   "price": 22900.0,
8   "discountType": null,
9   "discountedPrice": 22900.0
10 }
```

1.4 ลบ Product ใหม่

HTTP <http://localhost:8081/accounts> Save

DELETE <http://localhost:8080/products/4> Send

Params Auth Headers (8) **Body** Pre-req. Tests Settings Cookies

raw **JSON** Beautify

```
1 [{"id": "4", "name": "Ipad Air", "category": "Electronics", "brand": "Apple", "stock": 10, "price": 22900.0, "memberType": "NONE"}]
```

Body 200 OK 9 ms 38 B Save Response

Pretty Raw Preview Visualize **Text**

```
1
```

1.5 กรอง Product ตาม category

```
localhost:8080/products/category/Electronics

Pretty-print ☒

[
  {
    "id": "1",
    "name": "iPhone 15 Pro (673380605-0 sec3)",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 50,
    "price": 39900,
    "discountType": "MEMBER",
    "discountedPrice": 35910
  },
  {
    "id": "2",
    "name": "MacBook Air M3",
    "category": "Electronics",
    "brand": "Apple",
    "stock": 20,
    "price": 49900,
    "discountType": "NONE",
    "discountedPrice": 49900
  },
  {
    "id": "3",
    "name": "Samsung Galaxy S24",
    "category": "Electronics",
    "brand": "Samsung",
    "stock": 30,
    "price": 29900,
    "discountType": "SEASONAL",
    "discountedPrice": 23920
  }
]
```

1.6 ดูราคา Product

```
localhost:8080/products/1/price

Pretty-print ☐

35910.0
```

1. Reactive Programming vs Blocking ต่างกันอย่างไร

Blocking = 1 thread รอจนกว่าจะได้ผล ทำงานอื่นไม่ได้ระหว่างรอ

Reactive = ไม่รอ ไปทำงานอื่นก่อน พอข้อมูลพร้อมค่อย callback กลับมา ใช้ thread น้อยแต่รองรับ request เยอะ

2. Mono vs Flux ใช้กรณีไหน และทำไม

Mono<T> = คืนค่า 0-1 ค่า ใช้กับ findById, save, delete

Flux<T> = คืนค่า 0-N ค่า ใช้กับ findAll, findByCategory

3. WebClient — อธิบาย method chain .get(), .uri(), .retrieve(), .bodyToMono/Flux()

WebClient คือ HTTP client แบบ reactive ของ Spring ที่ใช้แทน RestTemplate (แบบ blocking) เดิม ใช้สำหรับเรียก REST API ภายนอกโดยไม่บล็อก thread ระหว่างรอ response method chain หลักมี 4 ส่วน

.get() / .post() / .delete()	กำหนด HTTP method ที่จะใช้ยิง request (GET, POST, PUT, DELETE) เป็นจุดเริ่มต้นของ chain ที่คืนค่าเป็น RequestHeadersUriSpec
.uri("/products/{id}", id)	กำหนด path ของ endpoint ที่จะยิงไป รองรับการใส่ path variable ผ่าน placeholder {id} แล้วแทนค่าด้วย argument ถัดไป ทำให้ปลอดภัยกว่าการต่อ string เอง (กัน URL injection)
.retrieve()	เริ่มขั้นตอนรับ response กลับมา เป็นจุดที่บอกว่า "เอาผลลัพธ์มาแปลงเป็น object เลย" (ถ้าต้องการ raw ClientResponse แบบละเอียด จะใช้ .exchangeToMono() แทน)
.bodyToMono(T.class) / .bodyToFlux(T.class)	แปลง response body (โดยปกติเป็น JSON) ให้เป็น Java object อัตโนมัติผ่าน Jackson — ใช้ bodyToMono เมื่อคาดหวังผลลัพธ์เดียว และ bodyToFlux เมื่อคาดหวังหลายรายการ (array)

4. Code ของ Repository, Service, Controller พร้อม comment อธิบาย

4.1 ProductRepository : Data Layer

```
public class ProductRepository {
```

```
private final Map<String, Product> store = new ConcurrentHashMap<>();
```

```
// ใช้ ConcurrentHashMap แทน HashMap ธรรมดา เพราะ Reactor รันงานบน
```

```
// หลาย thread พร้อมกัน (event loop) ต้อง thread-safe เพื่อกันข้อมูลเพี้ยน
```

```
// — 1. หา Product 1 รายการ —————
```

```
public Mono<Product> findById(String id) {
```

```
    Product product = store.get(id);
```

```
    // store.get() คืค่าแบบ synchronous (blocking นิดเดียว เพราะเป็นแค่ Map lookup ในหน่วยความจำ)
```

```
    // แต่ signature ของ method ต้องคืนเป็น Mono เพื่อให้ layer บนสุด (Controller)
```

```
    // ทำงานแบบ reactive ได้อย่างสม่ำเสมอทั้ง chain
```

```
    return product != null ? Mono.just(product) : Mono.empty();
```

```
    // ถ้าเจอ ห่อด้วย Mono.just() (ปล่อยค่าแล้วจบ)
```

```
    // ถ้าไม่เจอ คืน Mono.empty() (จบทันทีโดยไม่มีค่า — ไม่ใช่ error)
```

```
}
```

```
// — 2. หา Product ทั้งหมด —————
```

```
public Flux<Product> findAll() {
```

```
    return Flux.fromIterable(store.values());
```

```
    // แปลง Collection<Product> ธรรมดาให้กลายเป็น Flux
```

```
    // ที่ปล่อยค่าทีละตัวตามลำดับใน collection
```

```
}
```

// — 3. บันทึก Product —

```
public Mono<Product> save(Product product) {  
  
    store.put(product.getId(), product);  
  
    // side-effect: เขียนข้อมูลลง Map ก่อน (synchronous)  
  
    return Mono.just(product);  
  
    // แล้วค่อยห่อ product ที่บันทึกเสร็จแล้วด้วย Mono.just() เพื่อคืนกลับไป  
  
}
```

// — 4. ลบ Product —

```
public Mono<Void> deleteById(String id) {  
  
    store.remove(id);  
  
    return Mono.empty();  
  
    // Mono<Void> ไม่มี "ค่า" ให้คืนจริงๆ เพราะ delete เป็นแค่ side-effect  
  
    // การคืน Mono.empty() บอกคำว่า "งานเสร็จแล้ว" (onComplete) ไม่มี error  
  
}
```

// — 5. กรองตาม category —

```
public Flux<Product> findByCategory(String category) {  
  
    return findAll()  
  
        .filter(p -> p.getCategory().equalsIgnoreCase(category));  
  
    // reuse findAll() แล้วใช้ operator filter() กรองเฉพาะตัวที่ category ตรงกัน
```

```

        // equalsIgnoreCase กันปัญหาตัวพิมพ์เล็ก-ใหญ่ไม่ตรงกัน

    }

}

```

4.2 ProductService : Business Logic Layer

@Service

public class ProductService {

```

    private final ProductRepository repository;

    // Constructor Injection — Spring จะ inject ProductRepository bean เข้ามาให้เอง
    // ทำให้ class นี้ไม่ต้องสร้าง ProductRepository เอง (Dependency Inversion Principle)
    public ProductService(ProductRepository repository) {
        this.repository = repository;
    }

```

```

// ——— 1. ดึง Product 1 รายการ —————
public Mono<Product> getById(String id) {
    return repository.findById(id)
        .switchIfEmpty(Mono.error(new RuntimeException("Product not found: " + id)));

    // switchIfEmpty() คือจุดที่ต่างจาก repository.findById() ตรงๆ:
    // ถ้า repository คืน Mono.empty() (ไม่เจอ) ให้ "สลับ" ไปเป็น Mono.error() แทน
    // นี่คือที่มาของ business rule "ต้อง throw exception ถ้าไม่เจอ product"
    // ซึ่งเป็นหน้าที่ของ Service layer ไม่ใช่ Repository (แยก concern กันชัดเจน)
}

```

```

// ——— 2. ดึง Product ทั้งหมด —————
public Flux<Product> getAll() {
    return repository.getAll != null ? repository.findAll() : repository.findAll();

    // ในทางปฏิบัติเขียนแค่ return repository.findAll(); เพราะไม่มี logic เพิ่ม
    // Service เป็นแค่ทางผ่านสำหรับ endpoint นี้
}

```

```

// ——— 3. บันทึก Product —————

```

```

public Mono<Product> save(Product product) {
    if (product.getId() == null) {
        product.setId(UUID.randomUUID().toString());
        // ถ้า client ไม่ส่ง id มา (กรณีสร้างใหม่) ให้ generate UUID ให้เอง
        // กันปัญหา id ซ้ำ/ชนกัน ซึ่งเป็น business rule ที่ควรอยู่ที่ Service
        // ไม่ใช่ปล่อยให้ Controller หรือ Repository ทำ
    }
    return repository.save(product);
}

// ——— 4. ลบ Product —————
public Mono<Void> delete(String id) {
    return repository.delete(id);
    // ส่งต่อให้ Repository ตรงๆ เพราะไม่มี business rule เพิ่มเติมสำหรับการลบ
}

// ——— 5. กรองตาม category —————
public Flux<Product> getByCategory(String category) {
    return repository.findByCategory(category);
}

// ——— 6. คำนวณราคาหลังส่วนลด —————
public Mono<Double> getDiscountedPrice(String id) {
    return getById(id)
        .map(p -> p.getDiscountedPrice());
    // reuse getById() ที่มี error handling อยู่แล้ว (ไม่ต้องเขียนซ้ำ)
    // แล้วใช้ .map() แปลง Mono<Product> ให้กลายเป็น Mono<Double>
    // map() คือ transform แบบ synchronous (ไม่ต้องเรียก async operation เพิ่ม)
}
}

```

4.3 ProductController : REST API Layer

@RestController

@RequestMapping("/products")

// @RestController = @Controller + @ResponseBody รวมกัน หมายความว่า

// return value ของทุก method จะถูกแปลงเป็น JSON โดยอัตโนมัติ (ไม่ต้องเขียน view เอง)

// @RequestMapping("/products") กำหนด base path ให้ทุก endpoint ในคลาสนี้ขึ้นต้นด้วย /products

```
public class ProductController {
```

```
    private final ProductService service;
```

```
    public ProductController(ProductService service) {
```

```
        this.service = service;
```

```
    }
```

```
    // GET /products/{id} → Mono<Product>
```

```
    @GetMapping("/{id}")
```

```
    public Mono<Product> getById(@PathVariable String id) {
```

```
        // @PathVariable ดึงค่า {id} จาก URL มาใส่ในตัวแปร id
```

```
        return service.getById(id);
```

```
        // Controller ไม่มี business logic เอง แคร์รับ request → ส่งต่อ Service → คืนผล
```

```
    }
```

```
    // GET /products → Flux<Product>
```

```
    @GetMapping
```

```
    public Flux<Product> getAll() {
```

```
        return service.getAll();
```

```
        // ไม่มี path เพิ่มเพราะใช้ base path /products ตรงๆ
```

```
    }
```

```
    // POST /products → Mono<Product>
```

```
    @PostMapping
```

```
    public Mono<Product> save(@RequestBody Product product) {
```

```
        // @RequestBody บอกให้ Spring แปลง JSON ใน request body
```

```
        // เป็น Product object โดยอัตโนมัติผ่าน Jackson
```

```
        return service.save(product);
```

```
    }
```

```
    // DELETE /products/{id} → Mono<Void>
```

```
    @DeleteMapping("/{id}")
```

```
    public Mono<Void> delete(@PathVariable String id) {
```

```
        return service.delete(id);
```

```
    }
```

```

// GET /products/category/{category} → Flux<Product>
@GetMapping("/category/{category}")
public Flux<Product> getByCategory(@PathVariable String category) {
    return service.getByCategory(category);
}

// GET /products/{id}/price → Mono<Double>
@GetMapping("/{id}/price")
public Mono<Double> getDiscountedPrice(@PathVariable String id) {
    return service.getDiscountedPrice(id);
}
}

```

สรุปแนวคิดการแบ่ง layer: ทั้ง 3 layer นี้ยึดหลัก **Single Responsibility Principle (SOLID)** Repository จัดการแค่ data storage, Service จัดการ business rule (เช่น throw error ถ้าไม่เจอ, generate id), Controller จัดการแค่รับ-ส่ง HTTP request/response เท่านั้น ทำให้แก้ไข/ทดสอบแต่ละส่วนแยกกันได้โดยไม่กระทบส่วนอื่น